

MCP Server

Status: *feature-complete on dev (Phases 0–3 + hardening); not yet on prod.* `submit_work_record` and `scenario-planner` tools are planned but not built. Design + phase history: [docs/plans/mcp-server.md](https://docs.plans/mcp-server.md).

The MCP server exposes Arthound's canonical production data to external AI agents over the **Model Context Protocol**, scoped per-organization under RLS. It is mounted as an ASGI sub-app at `/mcp` inside the FastAPI app (`main.py`, via `mcp_server/`), using FastMCP in resource-server mode over the Streamable HTTP transport.

Data follows **PAW — Product → Asset → Work**. The Asset is the central, stable identity everything anchors to. An agent starts from `paw_v1_list_products` / `paw_v1_list_assets`, then `paw_v1_get_asset` for the rich, pre-joined record. The surface is read-heavy; the three writes create paper-trail records for humans and never mutate production state.

Connecting

- **Endpoint:** `http://<host>:8000/mcp/` (Streamable HTTP; the trailing slash matters — bare `/mcp` hits the SPA route).
- **Auth:** every request carries `Authorization: Bearer <api-key>`. The API key is the bearer token (no OAuth dance); FastMCP rejects unauthenticated requests.
- **Clients:** the MCP Inspector (`npx @modelcontextprotocol/inspector`), Claude Code (`claude mcp add --scope user --transport http arthound http://localhost:8000/mcp/ --header "Authorization: Bearer <key>"`), or any MCP client. (In the Claude Code VS Code extension, run the `add` in the integrated terminal, then start a new chat — servers load at session start.)

Identity & scoping (model A1)

An API key resolves to a dedicated **service-account member** (`member_role='agent'`) of exactly one org. The server (`lib/agent_auth.py`) mints a short-lived `authenticated` JWT for that principal and binds it for the request, so **every existing RLS policy applies unchanged** — an agent sees exactly what its org can see. There is no new Postgres role and no parallel policy surface; the agent is just another caller under the **User** identity described in the [README security model](#).

- **Org boundary** → RLS (the principal's membership). Cross-org access is impossible by construction.
- **Within-org boundary** → the credential's `scopes`, enforced in `mcp_server/context.py:tool_call`:
- `mode: read` (default) or `write`. Write tools require `mode: write`.
- `tools: ["*"]` (default) or an explicit tool-name allowlist.
- **Actor attribution:** every write records `actor_type='agent'`, `actor_ref=<credential id>`.
- **Audit:** `agent_access_log` records `authenticated` / `tool_call` / `denied` / `expired` / `revoked`.

Cross-org scoping is guarded by `scripts/test_agent_scoping.py` (identity, org-scoping, credential secrecy, revoke/expire lifecycle).

Managing keys

Offline service-role admin CLI, `scripts/agent_keys.py` :

```
python scripts/agent_keys.py issue --org-type studio --org-name "PowderStudios" --label "loc"
python scripts/agent_keys.py issue --org-type vendor --org-id <uuid> --label "x" --tools "p
python scripts/agent_keys.py list --org-type studio --org-id <uuid>
python scripts/agent_keys.py revoke --id <credential_id> # sets revoked_at + removes the
```

`issue` prints the raw key **once** (only its sha256 hash is stored). Revoke fully off-boards: the credential is marked revoked and the principal's membership row is deleted, so no token can resolve it.

Rate limiting

Per-credential in-process token bucket (`mcp_server/ratelimit.py`), enforced per tool call:

`MCP_RATE_LIMIT_PER_MIN` (default 120) sustained, `MCP_RATE_LIMIT_BURST` (default 30) burst; `0` disables. It is per-worker (not shared across processes) — a Redis-backed limiter is the future upgrade, tracked with the account-system rate-limiter work.

Errors & output

Tools raise an MCP tool error (`isError: true`) for: unknown/unscoped id, a read-only credential attempting a write, a tool outside the credential's allowlist, rate-limit exceeded, and — for writes — **refusal to persist an orphan** when a canonical asset can't be resolved. Tools return JSON in both the MCP **text** block and **structuredContent** (generic object schema), so both LLM clients and programmatic clients work.

Tool catalog (`paw_v1_*`)

Names are versioned with underscores for client compatibility. A breaking schema change ships as `paw_v2_*` alongside `paw_v1_*`, never in place.

Reads

Tool	Args	Returns
<code>paw_v1_list_products</code>	<code>limit?</code>	products in scope (<code>product_id</code> , <code>name</code>)
<code>paw_v1_get_product</code>	<code>product_id</code>	product fields + linked asset summaries
<code>paw_v1_list_assets</code>	<code>product_id?</code> , <code>status?</code> , <code>item_type?</code> , <code>limit?</code>	asset summaries (slots + PAW position)
<code>paw_v1_get_asset</code>	<code>asset_id</code>	full asset: slots, fields, product context, work rollup
<code>paw_v1_list_work</code>	<code>asset_id?</code> , <code>status?</code> , <code>limit?</code>	source work records
<code>paw_v1_get_work</code>	<code>work_id</code>	full work record + asset linkage
<code>paw_v1_get_estimates</code>	<code>link_id?</code>	effective estimate matrix (base/override cells)
<code>paw_v1_get_schedule</code>	<code>asset_id</code>	derived schedule (work timeline + warnings) — studio-only
<code>paw_v1_get_workflow_definition</code>	—	workflow steps + dependency edges
<code>paw_v1_get_asset_timeline</code>	<code>asset_id</code> , <code>limit?</code>	chronological event stream (review + dispatch lifecycle); <code>partial: true</code> until asset-change-capture lands

Writes (require `mode: write`)

Tool	Args	Effect
<code>paw_v1_flag_asset_risk</code>	<code>asset_id</code> , <code>risk_type</code> , <code>summary</code> , <code>severity?</code> , <code>evidence?</code>	creates an <code>asset_flags</code> record
<code>paw_v1_request_human_review</code>	<code>asset_id</code> , <code>subject</code> , <code>context?</code>	creates a <code>review_requests</code> record (escalation)
<code>paw_v1_propose_estimate_adjustment</code>	<code>proposed_estimate_days</code> , <code>reasoning</code> , <code>workflow_step_id?</code> , <code>variable_values?</code> , <code>asset_id?</code> , <code>current_estimate_days?</code>	creates an <code>estimate_adjustment_proposals</code> record

Write records surface to humans via `/api/agent-activity` (`routes/agent_activity.py`): per-asset on the Asset viewer's **Agent** tab (with status triage) and in the **Agent activity** widget on the studio/vendor home.

Deliberately absent / not yet built

- No `update_asset_status` / `modify_schedule` / `delete_work` (status flows through work submission, schedule derives from estimates + dependencies, work is superseded not deleted), no `raw_query` / SQL, no `bulk_*`.
- `submit_work_record` (Phase 4) — planned, blocked on work-actuals tracking (no ArtHound-native work-record write target yet).
- **Scenario-planner tools** — not exposed. The `scenario_*` tables are studio-scoped and would slot into the same RLS model; read tools (`list_scenarios` / `get_scenario`) are a straightforward future phase, while triggering generation is a heavier (cost/async) design decision.

Database

- `agent_credentials` — API key (hash) → org + principal member + scopes + expiry/revoke (migration `20260616000001`).
- `agent_access_log` — append-only audit (migration `20260616000001`).
- `asset_flags` , `review_requests` , `estimate_adjustment_proposals` — agent-write paper-trail records (migration `20260616000002`).
- `studio_members` / `vendor_members` — `member_role` CHECK widened to include `'agent'` .